

LABORATORY 3

DEVELOP APPLICATION WITH FREERTOS AND TOUCHGFX

1. PURPOSE

- Develop embedded applications using **FreeRTOS**.
- Program and build applications with the **TouchGFX framework**.
- Develop a **multi-functional embedded system application**, including:
 - o Interfacing external peripherals with the **STM32F4** microcontroller.
 - o Designing a graphical user interface (GUI) and implementing multitasking using **FreeRTOS** and **TouchGFX**.

2. PREPARATION

- Practical instruction materials, STM32F429I-DISC1 development kit, and a mini-USB cable.
- Required software: STM32CubeIDE 1.17.0, TouchGFX 4.24.1 or 4.25.0, and Hercules.

3. CONTENT

3.0. Getting Started with FreeRTOS

- Open **STM32CubeIDE** and create a new project.
- Configure necessary peripheral components:
 - o Set **PG13** and **PG14** as **Output** pins.
 - o Configure **RCC** and the **System Clock** so that the CPU runs at **90 MHz**.
 - o Set up **USART1** in **Asynchronous mode** at **115200 bps**.
- Configure **FreeRTOS** in the Middleware section:
 - o Select **CMSIS V2** as the interface.
 - o Add a new task named **myTask02**.
- Generate the source code and implement the functions for the two tasks:
 - o **defaultTask**
 - o **myTask02**
- Build the project, flash the firmware, and run the program on the **STM32F429I-DISC1** board.



```

295 /* USER CODE END Header_StartDefaultTask */
296 void StartDefaultTask(void *argument)
297 {
298     /* USER CODE BEGIN 5 */
299     /* Infinite loop */
300     for(;;)
301     {
302         osDelay(500);
303         HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_14);
304     }
305     /* USER CODE END 5 */
306 }

314 /* USER CODE END Header_StartTask02 */
315 void StartTask02(void *argument)
316 {
317     /* USER CODE BEGIN StartTask02 */
318     /* Infinite loop */
319     for(;;)
320     {
321         HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
322         HAL_UART_Transmit(&huart1, "Hello from user task\n", 21, 10);
323
324         osDelay(500);
325     }
326     /* USER CODE END StartTask02 */
327 }

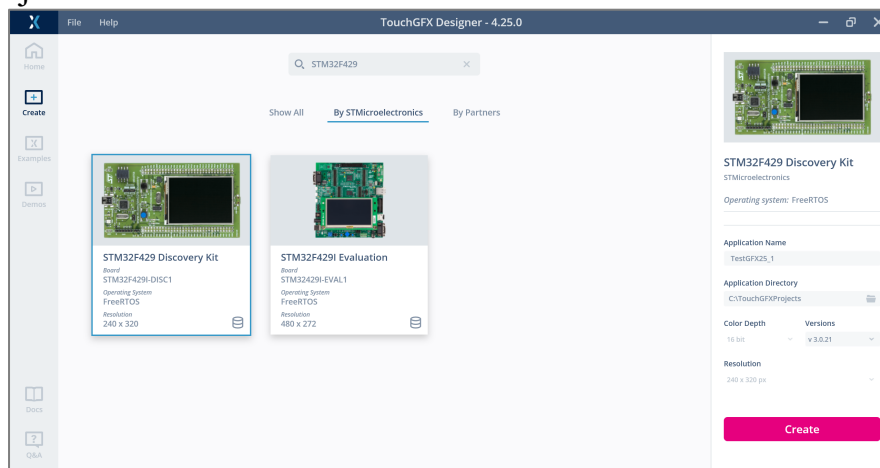
```

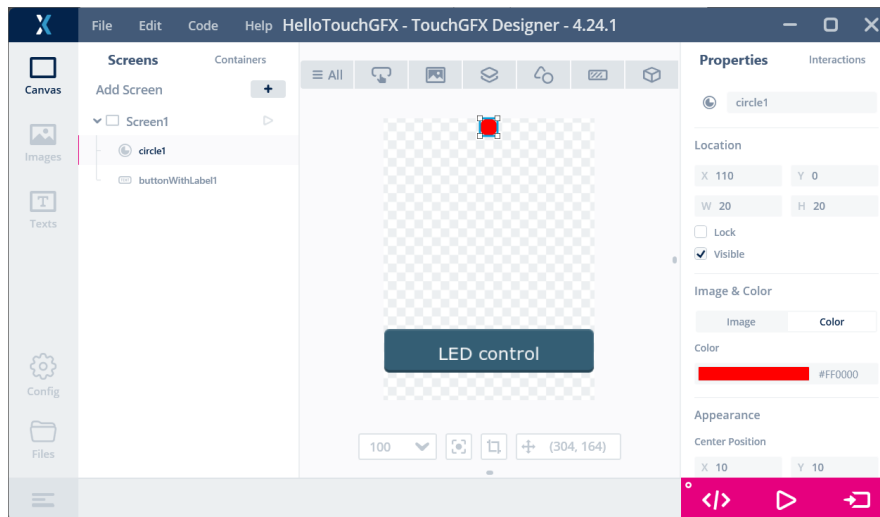
- Observe the program output:
 - The LED on pin **PG13** blinks at **1 Hz**.
 - The text **"Hello from user task"** is sent to the PC and displayed in **Hercules**.

3.1 Getting Started with TouchGFX and FreeRTOS

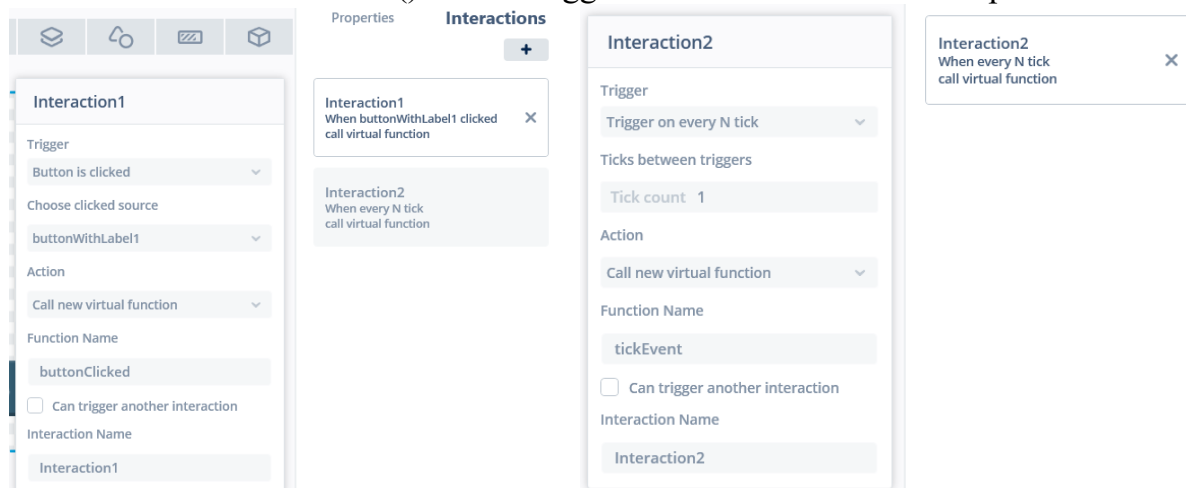
3.1.1 Creating the Project

- Launch TouchGFX Designer and create a new project for the STM32F429 board. Name the project HelloTouchGFX and make sure to use version 3.0.21.

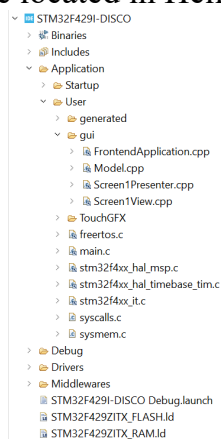




- In the design Canvas, add a Circle widget to Screen1. Adjust its size and color as shown in the reference, and set Radius = 10.
 - Add a Button with label to Screen1 and set its text to “LED control”.
 - Configure the button’s click event in the Interaction panel.
- The function `buttonClicked()` will be triggered whenever the button is pressed.



- Add and configure the tick event handler for Screen1. The `tickEvent()` function will be called every time the screen receives a time-update tick.
- Click Generate Code (F4) in the lower-right corner to create the project structure.
- Open STM32CubeIDE and import the generated project by double-clicking the .project file located in HelloTouchGFX/STM32CubeIDE.



- Build and flash the firmware to the STM32F429 board.

You should now see a red circle and a button displayed exactly as designed in TouchGFX.

- Explore and understand the structure of the generated project.

3.1.2. Programming Peripheral Integration and Event Handling

- In main.c, add the necessary GPIO initialization code for pins PG13, PG14, and PA0 (used for LEDs and the user button).

- Extend the **Screen1View** class with additional members:
 - Override the previously generated event-handling functions **buttonClicked()** and **tickEvent()** from the ViewBase class.
 - Add a local counter variable, e.g., **tickCount**, to track animation timing.

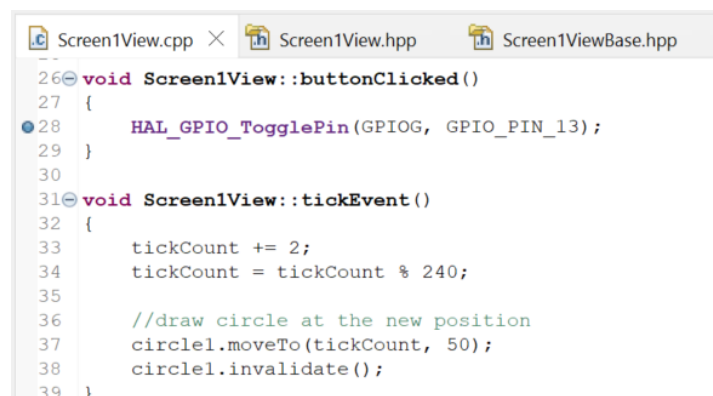
```
/* USER CODE BEGIN MX_GPIO_Init_2 */
/*Configure LED pins : PG13 PG14 */
GPIO_InitStruct.Pin = GPIO_PIN_13|GPIO_PIN_14;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

/*Configure GPIO pin : PA0 */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
/* USER CODE END MX_GPIO_Init_2 */
```

- Implement the **tickEvent()** function to update and animate the position of the circle dynamically on the display.
- Implement the **buttonClicked()** function to toggle the LED connected to **PG13** whenever the on-screen “LED control” button is pressed.

Note: Make sure to include:

- #include "touchgfx/widgets/canvas/Circle.hpp"



```
26 void Screen1View::buttonClicked()
27 {
28     HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_13);
29 }
30
31 void Screen1View::tickEvent()
32 {
33     tickCount += 2;
34     tickCount = tickCount % 240;
35
36     //draw circle at the new position
37     circle1.moveTo(tickCount, 50);
38     circle1.invalidate();
39 }
```

- Build and flash the program to the STM32F429 board, then observe the behavior:
 - The red circle moves horizontally across the screen in a repeating loop.
 - Pressing the **LED control** button toggles the PG13 LED (green LED) on the hardware.

3.1.3 Integrating the USER_BUTTON and Handling Events

- In main.c, declare a FreeRTOS queue inside the Private Variables section (right after the existing thread ID declarations).

```
/* USER CODE BEGIN PV */
osMessageQueueId_t myQueue01Handle;
const osMessageQueueAttr_t myQueue01_attributes = {
    .name = "myQueue01"
};
/* USER CODE END PV */
```

- Create the queue before creating any tasks in main.c.

```
/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
myQueue01Handle = osMessageQueueNew (16, sizeof(uint16_t), &myQueue01_attributes);

/* USER CODE END RTOS_QUEUES */
```

- In the default task, continuously poll the state of the USER_BUTTON and send the button state into the queue.

```
void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
        {
            uint8_t data = 'X';
            osMessageQueuePut(myQueue01Handle, &data, 0, 10);
        }

        osDelay(1);
    }
    /* USER CODE END 5 */
}
```

Modify the tickEvent() function in Screen1 View so it receives data from the queue and updates the animation accordingly.

Note: Make sure to declare extern myQueue01Handle;

```
void Screen1View::buttonClicked()
{
    HAL_GPIO_TogglePin(GPIOG, GPIO_PIN_13);
}

extern osMessageQueueId_t myQueue01Handle;
void Screen1View::tickEvent()
{
    tickCount += 2;
    tickCount = tickCount % 240;

    float x = tickCount / 55.0f;
    float y = sin(x)+sin(2*x)+sin(3*x)+1;

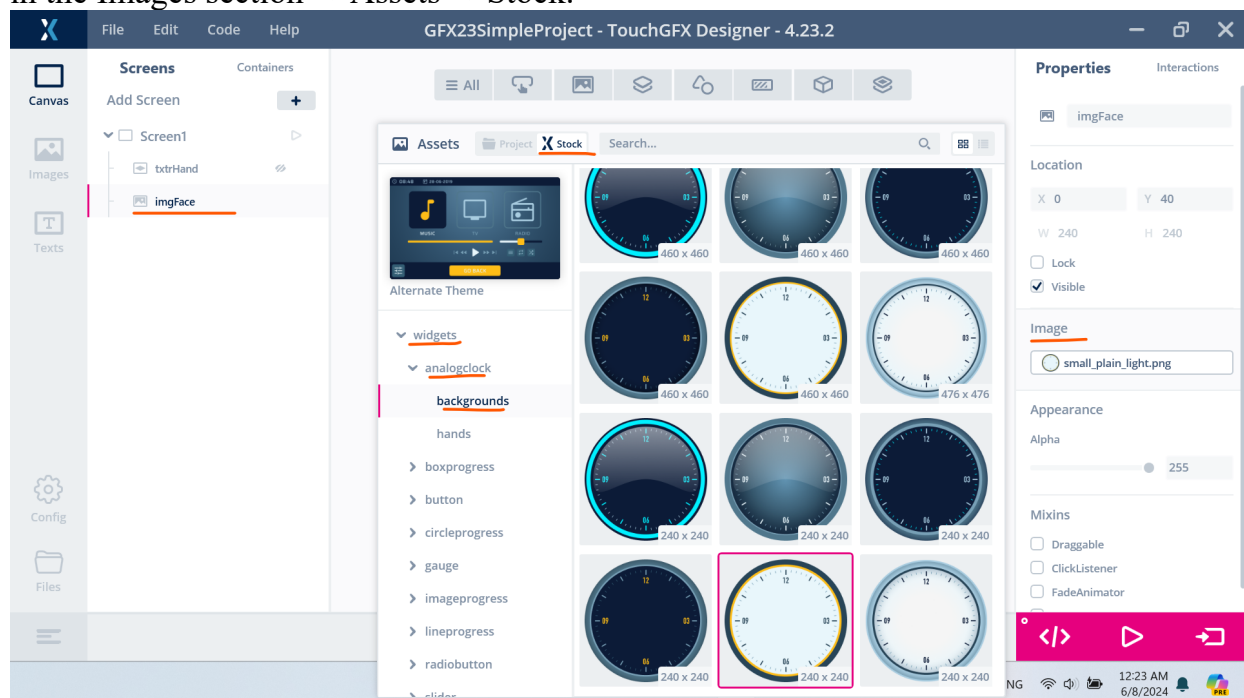
    uint8_t res;
    if (osMessageQueueGetCount(myQueue01Handle) > 0)
    {
        osMessageQueueGet(myQueue01Handle, &res, NULL, osWaitForever);
        if (res == 'X')
        {
            circle1.moveTo((int16_t)floor(x*55), 200-(int16_t)floor(y*50));
            circle1.invalidate();
        }
    }
}
```

- Build and flash the program to the STM32F429 board. Observe the system behavior:
- When the user presses the button, the circle moves from left to right while oscillating vertically across the screen.
- Provide an explanation of how the program works.

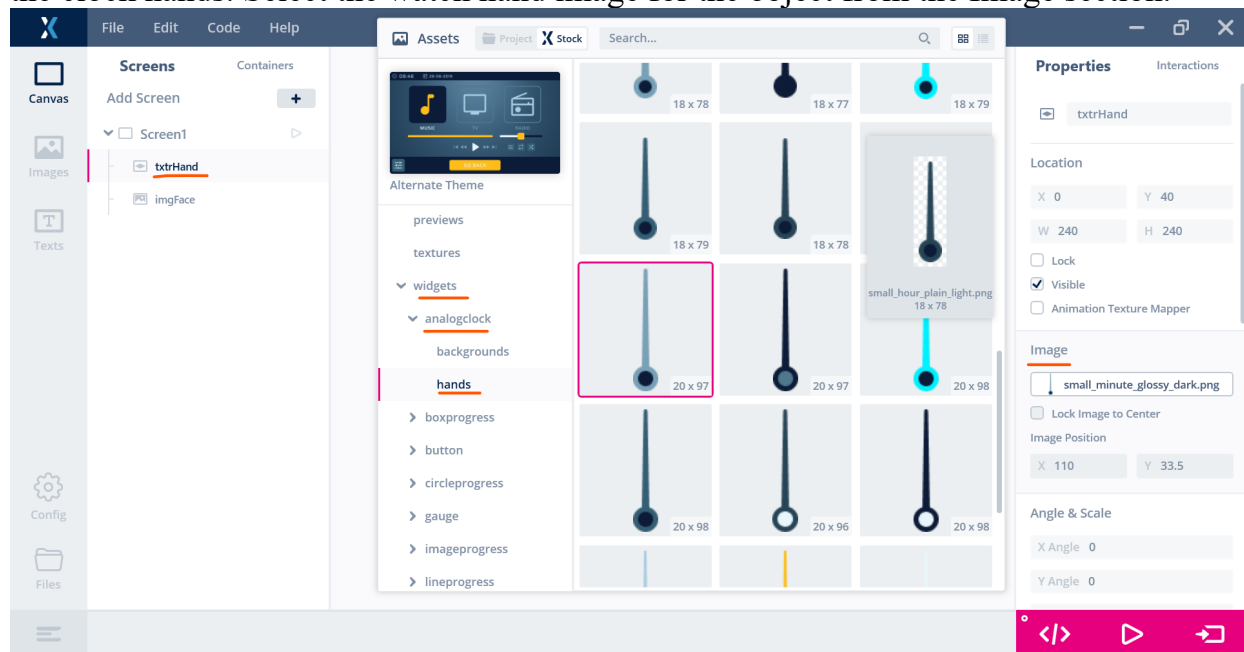
3.2 Building a Stopwatch Application

In this lab section, you will create a simple stopwatch interface using **TouchGFX Designer** and implement the stopwatch hand rotation on an **STM32F429** development board. You will also integrate a hardware push-button to start/stop the stopwatch.

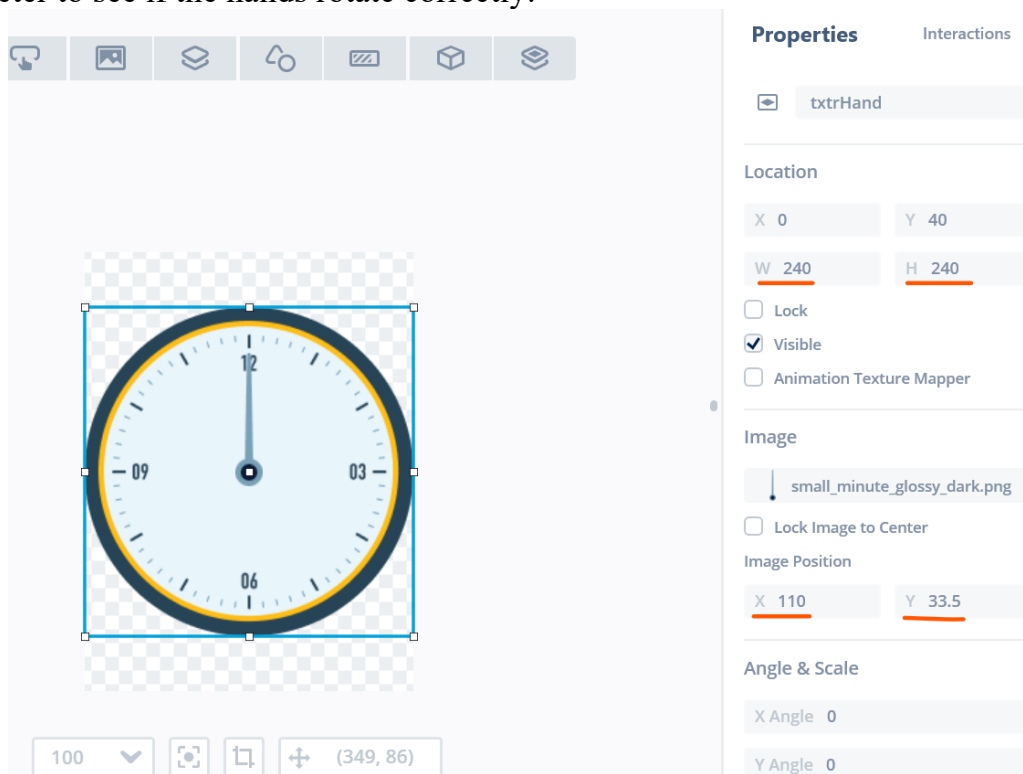
- Open TouchGFX Designer, create a new project with the STM32F429 board. Set the project name option.
- In the Canvas section for interface design, place an Image object (in the Images section) on Screen1 to display the clock face. Select the clock face image for the object in the Images section → Assets → Stock.



- Place another Texture Mapper object (also in the Images section) on Screen1 to display the clock hands. Select the watch hand image for the object from the Image section.



- Set the parameters for the txtrHand clock hands. You can try changing the Z Angle parameter to see if the hands rotate correctly.



- Press the red Generate code (F4) button in the bottom right corner.
- Double-click the .project file in the [Project folder]\STM32CubeIDE folder to import the project into STM32CubeIDE. Note: You need to delete the old project (but not the folder on the disk).
- Compile and upload the program to the STM32F429 kit. The result is a clock face with a single hand.
- Understand the project structure.
- Program the hand to move.

Add a member to Screen1View.

```

Screen1View.cpp  Screen1View.hpp x
1  #ifndef SCREEN1VIEW_HPP
2  #define SCREEN1VIEW_HPP
3
4  #include <gui_generated/screen1_screen/Screen1ViewBase.hpp>
5  #include <gui/screen1_screen/Screen1Presenter.hpp>
6
7  class Screen1View : public Screen1ViewBase
8  {
9  public:
10     Screen1View();
11     virtual ~Screen1View() {}
12     virtual void setupScreen();
13     virtual void tearDownScreen();
14     void handleTickEvent();
15     uint32_t tickCount;
16 protected:
17 };
18
19 #endif // SCREEN1VIEW_HPP
20

```

Handle the tick() event to update the clock hand position. Note that you should initialize tickCount to 0 in the Screen1View constructor.

```

void Screen1View::handleTickEvent()
{
    Screen1ViewBase::handleTickEvent();
    tickCount++;
    float rad = (tickCount % 360) * 3.14f / 180;

    //update and invalidate the clock hand
    txtrHand.updateZAngle(rad);
}

```

Translate and upload the program to the kit, then observe the movement of the clock hands.

Do it yourself: use the function

uint32_t osKernelGetTickCount (void)

Ref: [Kernel Information and Control \(arm-software.github.io\)](https://arm-software.github.io)

To obtain the system time, then calculate and update the hand movement angle based on the seconds parameter.

Note: #include <cmsis_os.h>

- Program the user button to turn the stopwatch on/off.

Configure the input/output pins in the MX_GPIO_Init() function in main.c

```

/*Configure GPIO pin : USER_BUTTON is A0 pin */
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

```

Declare the queue in the Private Variable section of the main.c file (after declaring the thread ID).

```

/* USER CODE BEGIN PV */
osMessageQueueId_t Queue1Handle;
const osMessageQueueAttr_t Queue1_Attributes = {
    .name = "Queue1"
};
osStatus_t r_state;
/* USER CODE END PV */

```

Create a queue before creating a task in the main.c file.

```

205  /* USER CODE BEGIN RTOS_QUEUES */
206  /* creation of Queue1 */
207  Queue1Handle = osMessageQueueNew (8, sizeof(uint8_t), &Queue1_attributes);
208  /* USER CODE END RTOS_QUEUES */

```

Continuously polling the button click status in the default task and sending the data to the queue.

```

void StartDefaultTask(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        if (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_SET)
        {
            uint32_t count = osMessageQueueGetCount(Queue1Handle);
            if (count < 2)
            {
                uint8_t data = 'P';
                osMessageQueuePut(Queue1Handle, &data, 0, 10);
            }
        }
        osDelay(1);
    }
}

```

Modify the handleTickEvent function of Screen1View to receive data from the queue and update the clock hand's behavior.

```

void Screen1View::handleTickEvent()
{
    static uint32_t prevTick = 0;
    Screen1ViewBase::handleTickEvent();
    tickCount++;
    float rad = 0;

    uint8_t res = 0;
    uint32_t count = osMessageQueueGetCount(Queue1Handle);
    if (count > 0)
    {
        osMessageQueueGet(Queue1Handle, &res, NULL, osWaitForever);
        if (res == 'P')
        {
            //update and invalidate the clock hand
            rad = ((tickCount - prevTick) % 360) * 3.14f / 180;
            txtrHand.updateZAngle(rad);
        }
    }
    else
    {
        prevTick = tickCount;
    }
}

```

Compile and flash the program on the kit. Observe the circuit's operation:

- When the button is not pressed, the clock hand remains stationary.
- When the button is pressed, the clock hand moves clockwise until the button is released.

Explain the program's operation.

3.3 Self-made exercise

Using the source code in the project “SimpleRacing”, build the interface and basic animation for a racing game.

- The track moves from top to bottom.
- The car can only move horizontally.
- Connect the hardware to control the car. You can choose 1 of 2 options (See the appendix for details):
 - o Make your own circuit with 2 buttons connected to pins PG2 and PG3 to control the car to move left/right. Note: PG2 and PG3 need to be configured in input mode with pull-up resistor.

- Use a joystick to control the car to move left and right. Only need to connect 1 ADC channel because you only need to control the car in 1 horizontal direction.
- Add features such as acceleration, random obstacle generation, scoring, saving and displaying High score.
- Write a report describing in detail the operation of the hardware + software in the game application.

Note: The sample project needs to be located in the C:\TouchGFXProjects folder.

APPENDIX:

The .ioc file is a configuration file used for automatic code generation, similar to TouchGFX. In this project, the code generation function from the .ioc file will not be reused to avoid overwriting manually modified code. Therefore, no new configuration or code generation from the .ioc file will be performed.

A.1 Configuring GPIO PG2 and PG3

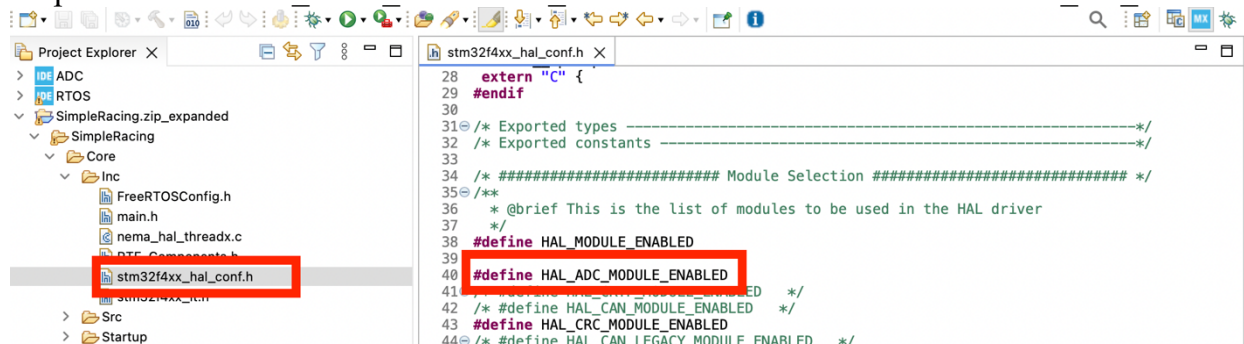
In the MX_GPIO_Init() function, configure the PG2 and PG3 pins in Input mode with pull-up resistors according to the code below.

```
__HAL_RCC_GPIOG_CLK_ENABLE();
/*Configure GPIO pin : PG2 and PG3 for input */
GPIO_InitStruct.Pin = GPIO_PIN_2 | GPIO_PIN_3;
GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOG, &GPIO_InitStruct);
```

A.2 ADC Configuration (GPIO PC3)

The SimpleRacing project does not include ADC configuration; therefore, you need to add the library and set up the ADC following these steps.

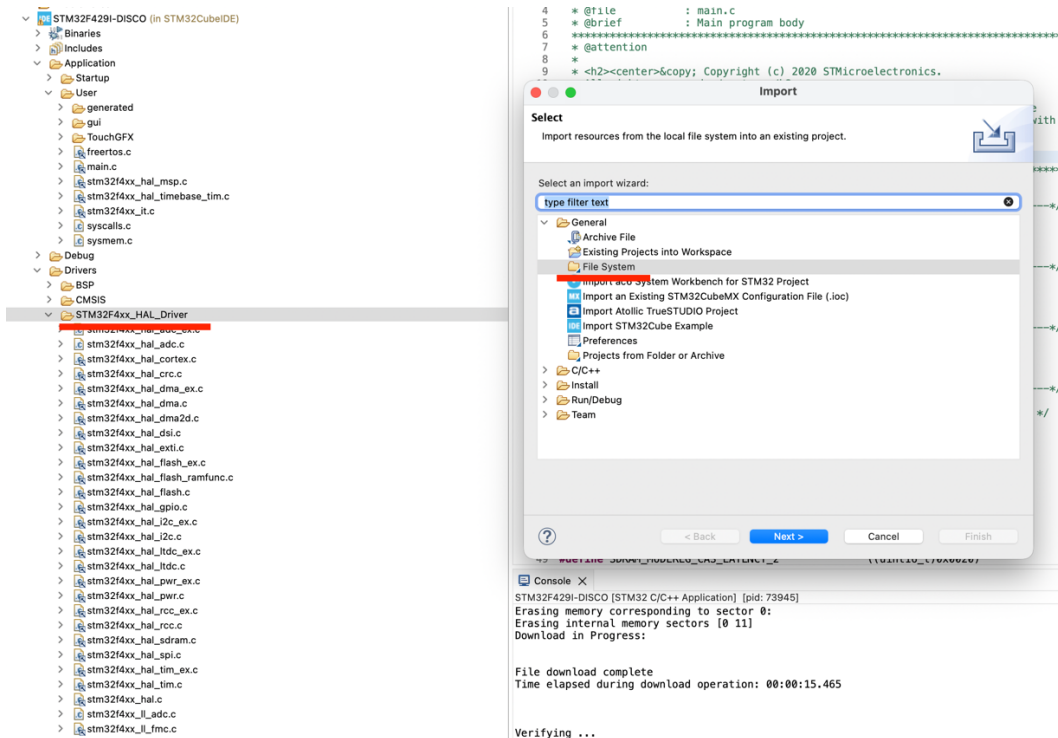
Step 1: Enable HAL_ADC_MODULE_ENABLED in /Core/Inc/stm32f4xx_hal_conf.h



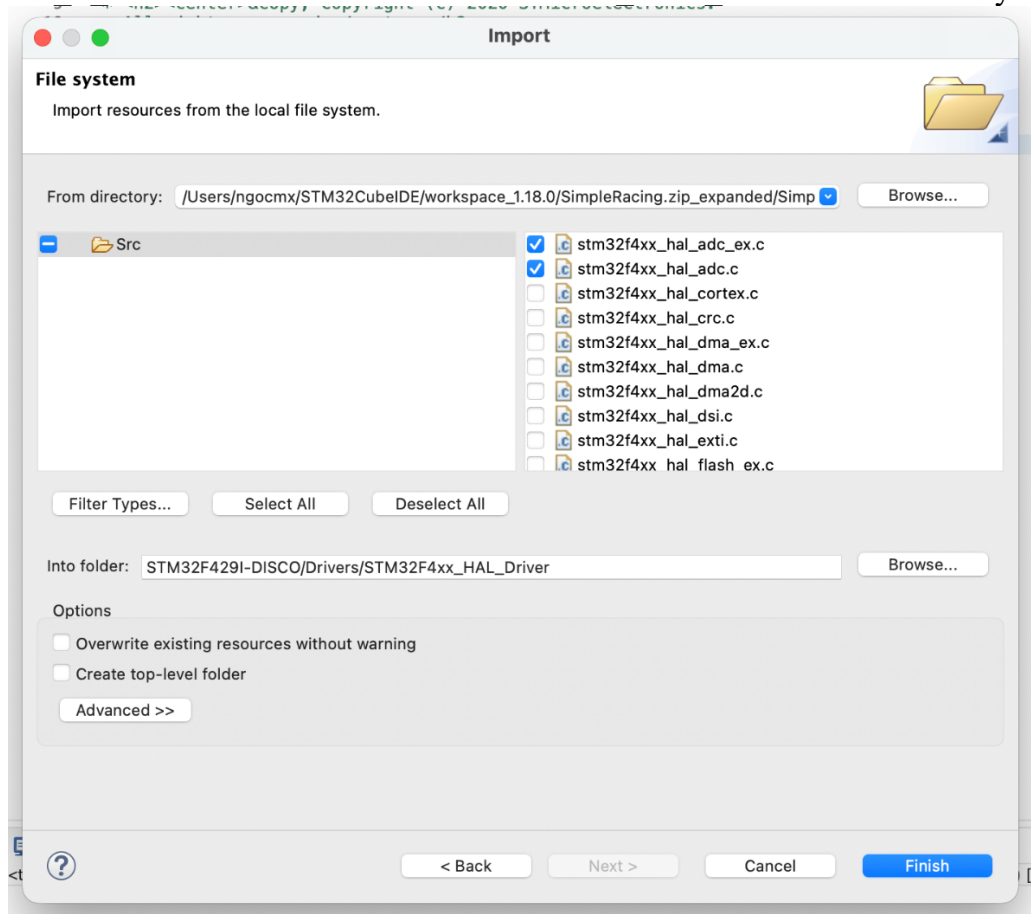
Step 2: Copy the ADC library files to the correct project folder

- Source files (.c): stm32f4xx_hal_adc.c, stm32f4xx_hal_adc_ex.c, stm32f4xx_ll_adc.c → vào thư mục Drivers/STM32F4xx_HAL_Driver/Src/
- Header files (.h): stm32f4xx_hal_adc.h, stm32f4xx_hal_adc_ex.h, stm32f4xx_ll_adc.h → vào thư mục Drivers/STM32F4xx_HAL_Driver/Inc/

Step 3: Import the ADC source files into the project by: Right-clicking the STM32F4xx_HAL_Driver folder, selecting Import, and then selecting File System.



Next, import the .c files `stm32f4xx_hal_adc_ex.c`, `stm32f4xx_hal_adc.c`, and `stm32f4xx_hal_flash_ex.c` from the `/Drivers/STM32F4xx_HAL_Driver/Src/` directory.



Step 4: Add the ADC configuration functions to the `stm32f4xx_hal_msp.c` file.

/**

* @brief ADC MSP Initialization

* This function configures the hardware resources used in this example

* @param hadc: ADC handle pointer

* @retval None

*/

void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc)

```
{
    GPIO_InitTypeDef GPIO_InitStructure = {0};
    if(hadc->Instance==ADC1)
    {
        /* USER CODE BEGIN ADC1_MspInit 0 */

        /* USER CODE END ADC1_MspInit 0 */
        /* Peripheral clock enable */
        __HAL_RCC_ADC1_CLK_ENABLE();

        __HAL_RCC_GPIOC_CLK_ENABLE();
        /**ADC1 GPIO Configuration
        PC3          -----> ADC1_IN13
        */
        GPIO_InitStructure.Pin = GPIO_PIN_3;
        GPIO_InitStructure.Mode = GPIO_MODE_ANALOG;
        GPIO_InitStructure.Pull = GPIO_NOPULL;
        HAL_GPIO_Init(GPIOC, &GPIO_InitStructure);

    }
}
```

/**

* @brief ADC MSP De-Initialization

* This function freeze the hardware resources used in this example

* @param hadc: ADC handle pointer

* @retval None

*/

void HAL_ADC_MspDeInit(ADC_HandleTypeDef* hadc)

```
{
    if(hadc->Instance==ADC1)
    {
        __HAL_RCC_ADC1_CLK_DISABLE();

        /**ADC1 GPIO Configuration
        PC3          -----> ADC1_IN13
        */
        HAL_GPIO_DeInit(GPIOC, GPIO_PIN_3);
    }
}
```

}

Step 5: Add the MX_ADC1_Init() function to main.c

```
/**
 * @brief ADC1 Initialization Function
 * @param None
 * @retval None
 */
static void MX_ADC1_Init(void)
{
    /* USER CODE BEGIN ADC1_Init 0 */
    /* USER CODE END ADC1_Init 0 */
    ADC_ChannelConfTypeDef sConfig = {0};
    /* USER CODE BEGIN ADC1_Init 1 */
    /* USER CODE END ADC1_Init 1 */
    /** Configure the global features of the ADC (Clock,
    Resolution, Data Alignment and number of conversion)
    */
    hadc1.Instance = ADC1;
    hadc1.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV4;
    hadc1.Init.Resolution = ADC_RESOLUTION_12B;
    hadc1.Init.ScanConvMode = DISABLE;
    hadc1.Init.ContinuousConvMode = DISABLE;
    hadc1.Init.DiscontinuousConvMode = DISABLE;
    hadc1.Init.ExternalTrigConvEdge =
    ADC_EXTERNALTRIGCONVEDGE_NONE;
    hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
    hadc1.Init.NbrOfConversion = 1;
    hadc1.Init.DMAContinuousRequests = DISABLE;
    hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
    if (HAL_ADC_Init(&hadc1) != HAL_OK)
    {
        Error_Handler();
    }
}
```

Thus, the configuration of ADC1 (channel PC3 – ADC1_IN13) is complete.

- END -